

Random Forest Algorithm

A Complete Step-by-Step Guide for Students

Dr. Anilkumar V. Brahmane | Computer Engineering | Sanjivani College of Engineering





What is a Random Forest? 🌳



Definition

A Random Forest is a collection of many Decision Trees working together to give one final answer.

Type

Supervised Machine Learning algorithm — used for both **Classification** and **Regression**.

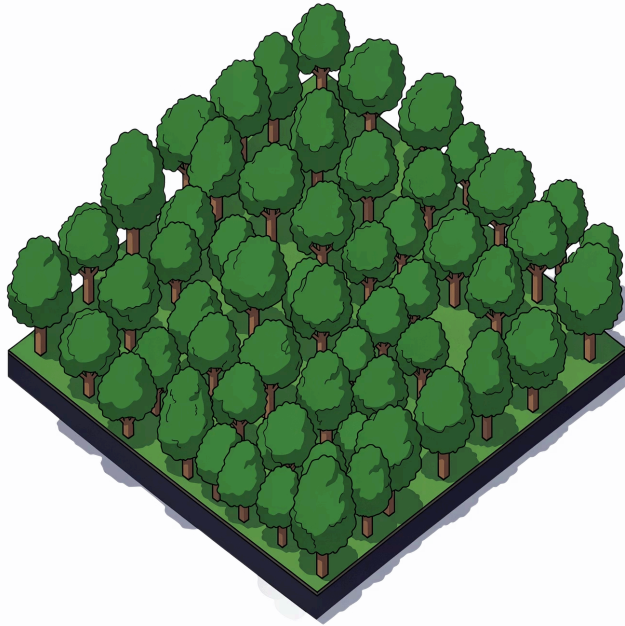
Simple Analogy 💡

Instead of asking one expert, you ask 100 experts and take the majority vote — **that's Random Forest!**

Introduced By

Leo Breiman in **2001**

Why is it Called "Random Forest"? 🤔



🌲 "Forest"

It builds many Decision Trees — and a group of trees is a forest!

🎲 "Random"

Randomness is used in **two powerful ways**:

- **Random rows** — Bootstrap Sampling picks different training rows for each tree
- **Random features** — Only a random subset of columns is considered at each split

📌 This randomness makes each tree different — and together they are **much smarter!**

Why Do We Need Random Forest? 🚀

⚠️ The Problem

A single Decision Tree **overfits** — it memorizes training data but fails on new, unseen data.

✅ The Fix

Random Forest combines many trees, **reducing errors** and improving accuracy dramatically.

💪 Handles Tough Data

Works well with **missing values**, **noisy data**, and very large datasets.

↻ Dual Purpose

Works for **Classification** (spam/not spam) and **Regression** (predicting house price).

Single Tree vs. Random Forest 🌱 vs 🌲 🌲 🌲

Key Insight

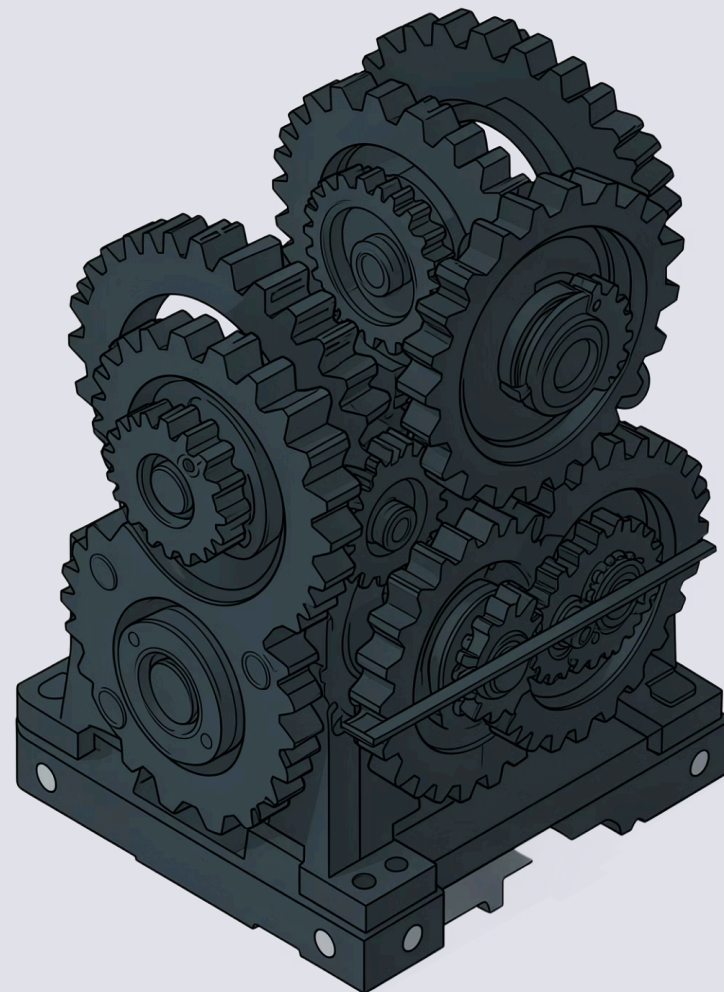
"Wisdom of the crowd" — many average models together beat one complex model. A lone Decision Tree is prone to overfitting and high variance. A Random Forest corrects this by aggregating hundreds of diverse trees into one robust prediction.

Feature	Decision Tree	Random Forest
Number of Trees	1	100s
Overfitting	High	Low
Accuracy	Moderate	High
Speed	Fast	Slower
Interpretability	Easy	Harder

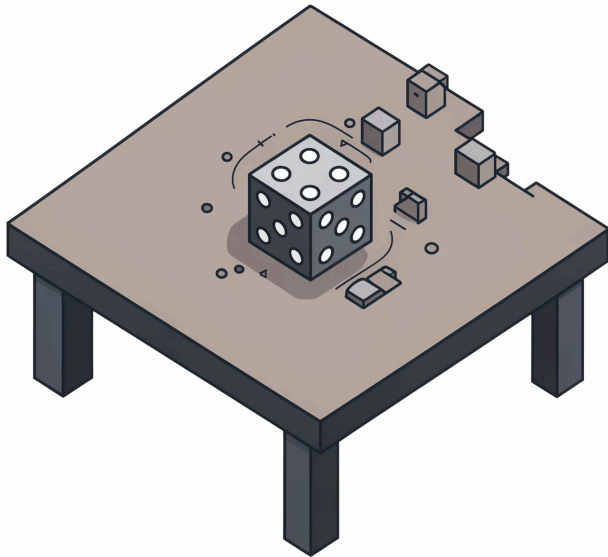
CHAPTER 2

How Random Forest Works — Step by Step

Five clear steps take you from raw data to a powerful, accurate prediction.
Let's break each one down.



Step 1: Bootstrap Sampling



What is Bootstrap Sampling?

From your original dataset of **N rows**, randomly pick N rows **with replacement** to create a new training set for each tree.

What does "with replacement" mean?

The same row **can be picked more than once** in a single sample — just like drawing a card, noting it, and putting it back.

 Example

Dataset has **1000 rows** → each tree trains on a random 1000-row sample (some rows repeated, some skipped). Each tree gets a **different random sample**, so every tree learns something slightly different!

Step 2: Random Feature Selection

1

At Every Split

Only a **random subset of features** (columns) is considered — not all features at once.

2

For Classification

Select $\sqrt{\text{total features}}$ features randomly at each node split.

3

For Regression

Select $\text{total features} \div 3$ features randomly at each node split.

4

Why? Diversity!

If one feature dominates, not every tree will use it — ensuring trees are **diverse and independent**.



Example: 9 features total → each split considers only **3 random features** ($\sqrt{9} = 3$)

Step 3: Build Multiple Decision Trees 🌲

How Many Trees?

Repeat Steps 1 & 2 to build **N independent Decision Trees** — typically **100 to 500 trees**.

Each Tree is Unique

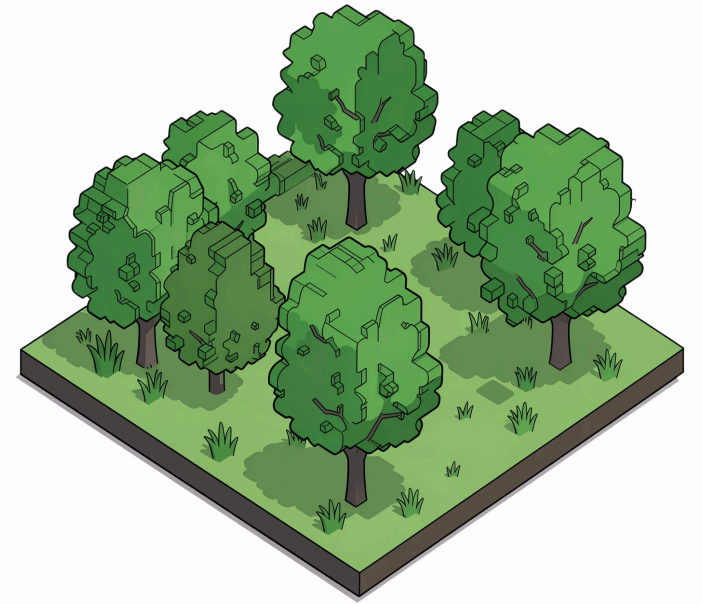
Every tree is trained on a **different bootstrap sample** with **different random features** at each split — giving each tree a unique perspective on the data.

Grown to Full Depth

Trees are grown without pruning. Each individual tree may overfit slightly, but together they **balance each other out**.

🌳 Result

A diverse "**forest**" of **trees**, each with a different view on the data — ready to vote together!



Step 4: Voting (Classification)

1

New Data Point

An unseen input is passed through every tree in the forest.

2

Each Tree Votes

Every tree independently gives its own prediction (e.g., Loan Approved / Rejected).

3

Majority Wins

The class predicted by the **most trees** becomes the final answer.

4

Final Answer 

Example: 100 trees → **70 say Approved**, 30 say Rejected →
Final: Approved!

Majority voting reduces the chance of any single wrong tree affecting the final result — the forest is wiser than any one tree.

Step 5: Averaging (Regression)



When do we Average?

For **regression tasks** (predicting a number rather than a class), each tree gives a numeric prediction.

Formula

Final Prediction = Average of all tree outputs

📌 Example: House Price Prediction

- Tree 1 predicts: ₹50L
- Tree 2 predicts: ₹55L
- Tree 3 predicts: ₹45L

→ Final Prediction: ₹50L (average of all trees)

- ❏ Averaging **smooths out** extreme predictions from individual trees, giving a stable and reliable output.



Step 6: The Complete Working Diagram



This end-to-end flow is the complete Random Forest pipeline — from raw data to a final, reliable prediction through the power of many trees working together.

Why is Random Forest Better than a Decision Tree?



Less Overfitting

Many trees cancel out each other's individual errors — the forest self-corrects.



Robust to Outliers

A few bad data points don't ruin the model — the majority vote protects it.



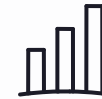
Higher Accuracy

An ensemble of trees outperforms a single tree on almost every real-world dataset.



Works Out-of-the-Box

Needs very little tuning compared to other algorithms — great for beginners!



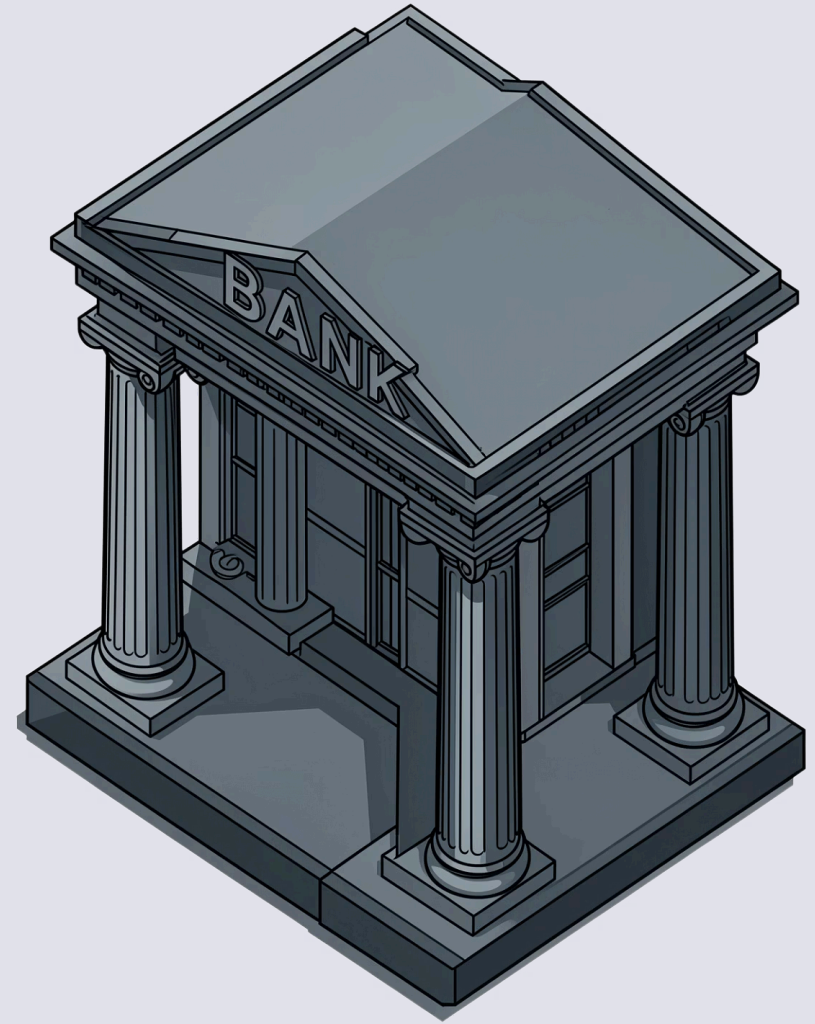
Feature Importance

Tells you which features matter most — invaluable for understanding your data.

CHAPTER 3

Application: Loan Approval Prediction

Let's apply everything we've learned to a complete, real-world case study using Python and scikit-learn.



Problem Statement: Loan Approval

Goal

Predict whether a bank should **Approve**  or **Reject**  a loan application automatically.

Input Features

- Age
- Monthly Income
- Credit Score
- Loan Amount
- Employment Status
- Existing Debts

Output

Approved (1) or Rejected (0)

-  Why Random Forest? Multiple factors affect loan decisions — Random Forest handles many features and complex patterns very well!



Implementation: Step 1 — Data Preparation

01

Collect Data

Gather historical loan records with known outcomes (approved / rejected).

03

Encode Categories

Convert text labels to numbers: "**Employed**" → 1, "**Unemployed**" → 0.

02

Handle Missing Values

Fill missing entries with **mean/median** values, or drop incomplete rows.

04

Split the Data

80% Training Set to teach the model, **20% Testing Set** to evaluate it.

Implementation: Step 2 — Train the Model

Python Code

```
# Step 1: Import the library
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

# Step 2: Split data (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 3: Create the model
model = RandomForestClassifier(
    n_estimators=100, # Build 100 trees
    random_state=42  # For reproducibility
)

# Step 4: Train the model
model.fit(X_train, y_train)
```

Key Parameters Explained

- **n_estimators=100** → Build 100 Decision Trees in the forest
- **random_state=42** → Ensures results are reproducible every time you run
- **model.fit()** → This is where the learning happens!

📌 More trees = more accuracy, but also more computation time. 100–200 trees is a great starting point.

Implementation: Step 3 — Predict & Evaluate

Python Code

```
# Predict on test data
y_pred = model.predict(X_test)

# Check accuracy
from sklearn.metrics import accuracy_score,
confusion_matrix
acc = accuracy_score(y_test, y_pred)
print(f"Accuracy: {acc * 100:.2f}%") # e.g., 92%

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)

# Feature importance
importances = model.feature_importances_
print("Feature Importances:", importances)
# Result: Credit Score is most important!
```

What Do These Metrics Tell Us?

- **accuracy_score** → Overall % of correct predictions (e.g., 92%)
- **confusion_matrix** → Shows True Positives, False Positives, True Negatives, False Negatives
- **feature_importances_** → Which features had the most impact on predictions

✅ A 92% accuracy means the model correctly predicts loan outcomes 92 out of every 100 times! 🎯

Loan Approval — Sample Output

Applicant Profile

- Age: **35 years**
- Income: **₹60,000/month**
- Credit Score: **750**
- Loan Amount: **₹5 Lakhs**

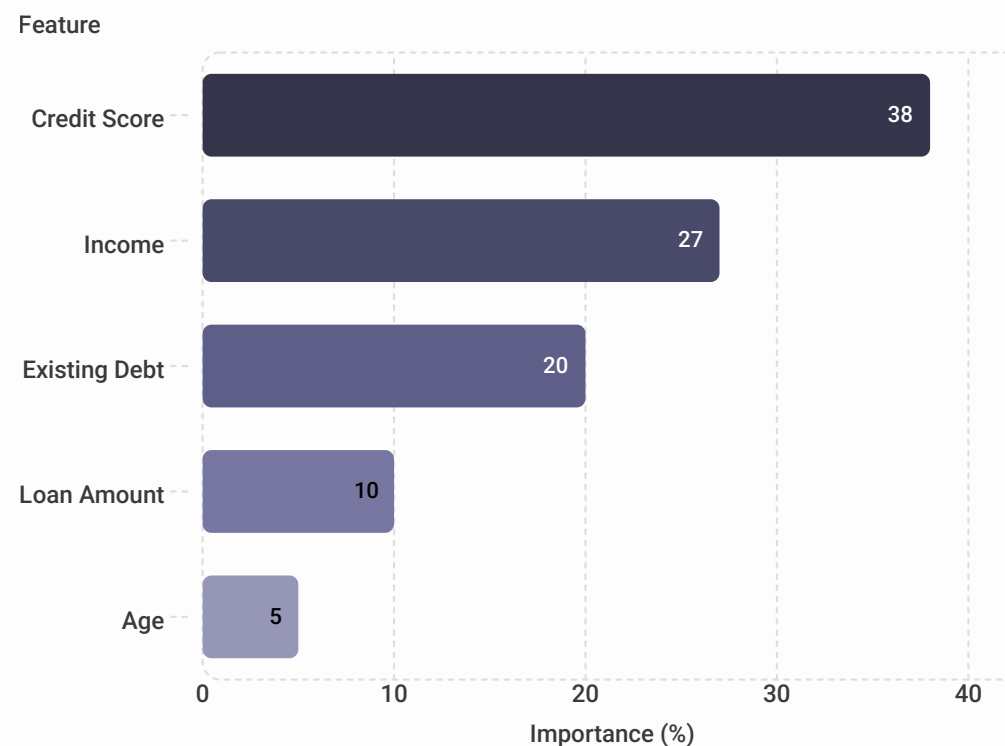
Forest Votes

100 trees vote → **85 say Approved**, 15 say Rejected

Final Decision

Loan Approved (Majority = 85%)

Feature Importance for Loan Approval



Key Takeaways



What is Random Forest?

Many Decision Trees + Voting (Classification) or Averaging (Regression) = one powerful model.



Why "Random"?

Random rows via Bootstrap Sampling + Random features at each split — diversity is the secret!



Why Better?

Less overfitting, higher accuracy, feature importance, and robust to bad data points.



Real-World Use Cases

Loan approval, fraud detection, medical diagnosis, stock price prediction — and many more!



Remember This Forever

"No single tree is perfect, but a forest is wise!"